

Synthèse Java - Juin 2020

Partie 1

1. Expliquer pourquoi l'on peut dire que Java est un langage orienté objets, basé sur C et C++, de haut niveau, très typé, simple, robuste, adapté au réseau et aux threads, sécurisé, portable.

- **la simplicité** : par rapport au C++, dont il ne peut nier une certaine filiation, Java n'utilise pas les pointeurs, la récupération de l'espace mémoire alloué dynamiquement (il est récupéré automatiquement), la surcharge des opérateurs, l'héritage multiple.
- **la robustesse** : l'absence de pointeurs et l'obligation de définir sans ambiguïté les variables utilisées rend impossible des accès involontaires à des zones mémoire protégées ou inaccessibles.
- **l'adéquation à l'environnement réseau** : l'usage des protocoles de la mouvance TCP/IP (http, ftp) se réalise très simplement par l'utilisation des URLs.
- **la gestion du multithreading** : ceci rend possible la prise en charge de problèmes concurrentiels, la seule limite étant celle de la machine; Mais, surtout, les deux qualités essentielles qui expliquent le succès de Java sont
- **la portabilité** : le code généré par le compilateur Java est exécutable sur tout environnement disposant de l'interpréteur Java approprié; de plus, la programmation Java ne connaît pas des instructions faisant explicitement référence à la taille de la représentation des divers types de données
- **la sécurité** : tout est prévu pour que le transfert d'une applet (unité de code téléchargeable) ou l'exécution d'une application ne puisse endommager le site d'exécution.

2. Expliquer les étapes de développement et de déploiement (sur deux plateformes différentes) d'une application Java portable. En particulier, expliquer ce qu'est du bytecode et une machine virtuelle Java. Situer tous ces outils utilisés dans les répertoires obtenus en installant un JDK. Pourquoi certains de ces outils se trouvent-ils dans un répertoire nommé JRE ?

Le code généré par un compilateur Java est encore appelé du **bytecode** : ses instructions sont exprimées au niveau **le plus bas** possible sans devenir dépendant de la machine d'exécution.

L'objectif de portabilité est donc bien clair. On dit encore que ce code est destiné à une machine virtuelle Java, c'est-à-dire une application qui émule une machine architecturée sur un processeur RISC virtuel et pour laquelle le bytecode sera transformé par un interpréteur Java.

- **des outils de développement (dans un répertoire bin)** : le compilateur (l'exécutable javac), l'interpréteur (l'exécutable java), le générateur de documentation (l'exécutable javadoc), etc;
- **l'environnement d'exécution** (le plus souvent dans un répertoire jre), le JRE (Java Runtime Environment) qui permet l'exécution du bytecode Java, mais pas son développement : on y trouve donc tout ce dont a besoin une application Java pour s'exécuter, par exemple pour Win32 ou pour Solaris; on y trouve donc la machine virtuelle (jvm.dll sous Windows), le vérificateur (verify.dll), l'interpréteur java (mais donc pas le compilateur), etc et aussi les librairies (comme jawt.lib) et fichiers nécessaires (fichiers properties, policy, gif, ...); le sous-répertoire lib contient les fichiers class de la librairie standard Java, le plus souvent dans des fichiers compressés de type jar, comme par exemple rt.jar (run time);
- **les librairies nécessaires aux outils de développement** (dans le répertoire lib), sous forme de fichiers lib ou jar;
- **éventuellement, les fichiers sources** (donc écrits en Java) contenant toutes les classes faisant partie de l'API de base (fichier src.jar);

3. Expliquer qui sont ces personnages

James Gosling : fondateur de java, anciennement appelé Oak, il était employé chez Sun.

Sun System : Entreprise créatrice de JAVA

Eclipse : IDE Pour développer du code JAVA

JCP (Java Community Process) : est une organisation créée par Sun en 1998 regroupant des utilisateurs (particulier et grosses entreprises) visant à coordonner l'avancé des fonctionnalités de Java

4. Expliquer les bases de sécurité de la plate-forme Java : compilateur unique, vérificateur, chargeur de classe, sandbox, jars.

- **Sandbox** - Ensemble des fonctionnalités et d'objets qui assure la sécurité des applications. Son composant principal est le gestionnaire de sécurité. Par exemple, Il empêche par défaut à une applet d'accéder aux ressources du système
- **Vérificateur** - Celui-ci traite le bytecode dès sa réception par la machine cible en en vérifiant le format et en y recherchant tout élément susceptible d'être dangereux, comme des accès par référence trop curieux, des accès interdits

à des ressources, des dépassements de piles, des conversions illicites, des redéfinitions, etc

- **Chargeur de classe** - "charge le code" pour le mettre à la disposition de l'interpréteur (en clair, il "mappe" les références en adresses véritables). Mais aussi partitionne l'ensemble des classes Java intervenant en des sous-ensembles appelés des namespaces.
- **Jar** - L'utilitaire jar.exe permet de rassembler dans un même fichier d'extension jar un certain nombre d'autres fichiers qui sont typiquement
 - ◆ des fichiers bytecodes .class;
 - ◆ des fichiers .xml ou .html;
 - ◆ des fichiers images .gif ou .jpg.
 - ◆ etc. L'intérêt d'utiliser jar est que l'on peut ainsi produire un fichier :
 - ◆ comprimé : il prend donc moins de place sur le disque ou sur le réseau; le format de compression est le format ZIP;
 - ◆ sécurisé : il est possible d'y intégrer une signature électronique.

PARTIE 2

1. Expliquer ce que sont les charsets et les encodings. Pourquoi ces notions sont-elles si importantes en Java ?

- **Le charset** (Jeu de caractères) est une liste précise de caractères (Ex : ASCII, UNICODE)
- **L'encoding** est l'encodage d'un charset qui spécifie comment ces caractères seront stockés sous forme d'octets compréhensible pour la machine (Ex : UTF-8, UTF-16)
- Java a utilisé au départ l'ASCII mais il manquait des lettres pour les européens, on est ensuite passé à l'ISO Latin-1 mais il manquait de portabilité, on a donc utilisé l'UNICODE (charset sur 16 bits) Java utilise l'UTF-8 car il permet de tenir compte de l'inefficience d'UNICODE pour les codes ASCII courants. En gros, les codes ASCII sont codé sur un seul byte alors que les autres sont codés sur 2 à 4 byte.

2(A compléter). Comment et où les objets Java sont-ils créés et manipulés ? Expliquer les implications particulières pour les tableaux à 1 et 2 dimensions. Selon quels mécanismes la disparition d'un objet est-elle gérée ?

L'instanciation d'une classe se réalise de manière dynamique en utilisant l'opérateur new et un constructeur de la classe.

On l'a dit, il n'est pas nécessaire et, à vrai dire, pas possible de libérer de l'espace mémoire de sa propre initiative : c'est le rôle du **Garbage Collector** de réaliser cette libération. Quand agit-il ? Lorsque bon lui semble ... c'est-à-dire quand le système le réclame – on ne peut donc faire aucune supposition sur un espace mémoire libéré ou "pas encore libéré" !

3. Quel est le rôle de la classe Object ? Décrire les 3 mécanismes fondamentaux qu'elle fournit et expliquer pourquoi certains doivent être modifiés. Quel rapport avec le fait que les méthodes de Java sont "virtuelles" au sens du C++ ?

La classe Object est la mère de toutes les classes, Si on ne précise pas une super-classe, on suppose qu'une classe hérite de Object. Elle est composée de plusieurs méthodes dont :

- **Equals** qui vérifie l'égalité de 2 objets (Vérifie la valeur de ses membres), par défaut les seuls objets égaux à un objet c'est lui-même, on doit donc redéfinir cette méthode.
- **getClass** qui renvoie un objet instance de class qui peut contenir son nom (getName), Cette méthode ne peut être redéfinie car elle est final. Grâce à cette méthode il est possible de créer une nouvelle instance d'une classe sans connaître la nature de celle-ci.
- **ToString** qui permet la représentation d'un objet sous forme d'un string, il faut la redéfinir car le résultat espéré n'est pas bon.

4. Expliquer comment la comparaison des chaînes de caractères constantes (classe String) est gérée en Java.

En effet, on se souviendra qu'une chaîne de caractères constante (ce que l'on appelle encore un "littéral") est automatiquement considéré comme une instance de String.

De plus, le compilateur appelle automatiquement la méthode ***public native String intern()*** ;

Pour cet objet. Ceci a pour effet de placer, lors de l'exécution, la chaîne de caractères dans une zone des chaînes constantes, si cette chaîne ne s'y trouve pas déjà.

La zone est commune à tous les composants de l'application. L'intérêt de cette manière de faire est évident : une chaîne constante ne sera jamais représentée qu'une seule fois : il s'agit donc bien d'une optimisation.

Cependant, cette optimisation a un effet pervers quant à la comparaison de ces chaînes.

5. Expliquer à partir d'un exemple ce que sont les packages Java et selon quels principes ils sont retrouvés lors de la compilation et de l'exécution d'une application. En particulier, quel est le rôle de import, CLASSPATH, -classpath, public dans le processus de reconnaissance et d'exécution d'une méthode ?

- Un Package est un espace de définitions. Ça permet qu'un nom soit connu à l'intérieur de son package (namespace) et sera utilisé à l'extérieur préfixé du nom de son package. Cela permet d'éviter les conflits d'identificateur. Un package doit être organisé comme un répertoire, un package doit se trouver dans un répertoire portant le même nom que lui.
- L'accès package possède les mêmes droits que protected, en absence de qualification d'accès c'est l'accès package qui est utilisé par défaut. Faut veiller à ce que les classes instanciées soient publiques et qu'elles doivent se trouver dans un fichier portant leur nom.
- Un CLASSPATH est une variable d'environnement qui contient le nom des répertoires dans lesquels le compilateur et l'interpréteur doivent chercher les classes du package. Elle peut également contenir des fichiers de compression *.zip ou *.jar. Par défaut on y trouve le répertoire contenant les fichiers class des classes standard de Java. Les classes d'un package peuvent être utilisées
 - En préfixant le nom du package mais c'est lourd.
 - Soit en utilisant l'instruction import

Partie 3

1. Expliquer ce qu'est un interface Java, notamment par rapport à une classe abstraite. Expliquer et illustrer par un exemple ses deux rôles principaux en ce qui concerne la généricité des applications Java.

Une interface est une collection de déclaration de méthodes et de constantes, on y trouve des prototypes de méthode sans implémentation (On pourrait dire qu'il s'agit d'une classe abstraite ou aucune méthode est implémenté. **Le but d'une interface est de définir un protocole de comportement pour des classes quelconques.**

- **RÔLE 1**
Permet de définir une interface commun à des classes qui ne font pas partie de la même hiérarchie. Toutes classes qui acceptent ce protocole s'engage à implémenter les méthodes déclarés. La notion d'interface permet de contenir le problème des héritages multiple (Cela ne remplace pas l'héritage multiple.)
- **RÔLE 2**
On peut créer des listes/tableaux/vecteurs,... d'objets qui implémentent une même interface.

2. Java permet de définir des interfaces "vides". Donner des exemples et expliquer l'intérêt de cette possibilité.

On peut créer des listes/tableaux/vecteurs,... d'objets qui implémentent une même interface.

3. Expliquer le mécanisme de la "classe à la volée" implémentant un interface. Comment peut-on en voir la trace dans les fichiers bytecode ?

Une classe à la volé est une classe sans nom que l'on défini sur pièce comme étant une implémentation d'une interface

Exemple : On crée une interface, on « instancie » l'interface (XXX x = new XXX{ méthode de l'interface})

On a donc créé « à la volée » une classe sans nom mais que l'on définit sur pièce comme étant une implémentation de l'interface XXX.

Ce code génère 2 fichiers .class, un fichier pour la classe interne (la classe où se trouve la classe à la volé et un second pour la classe externe qui n'a pas de nom (la classe à la volé), le \$ sépare les 2 noms mais elle n'en a pas.

```

public interface passager
{
    Get passager() ;
}

```

```

Public class Test
{
    Public static void main
    {
        Passager A = new Passager()
        {
            Public object[] get passager
            {
                .....
            }
        };
    }
}

```

A implémente Passager. On crée une classe qui implémente l'interface passager, on lui donne la méthode get de l'interface. On a donc une classe sans nom qui implémente passager. Test est une classe public, cela va donner un fichier .class qui donnera Test.class. ici on a une 2ème classe imbriquée ca va donc donner un 2ème fichier .class car on ne peut avoir qu'une seule classe dans un fichier bytecode.

Quand on a une classe A et à l'intérieur une classe B, on aura A.class et A\$B.class (elle fait partie de la classe A). Dans notre cas, la 2 -ème classe a pas de nom donc on aura Test.class et Test\$1.

Classe sans nom = numéro 1...

4. Expliquer les deux manières de programmer correctement l'appel d'une méthode susceptible de lancer une exception "checkée" de manière à réussir la compilation.

- **La classe Throwable** est la classe mère de toutes les exceptions et erreurs, elle hérite elle bien évidemment de la classe Object.
- **La classe Error** hérite de la classe Throwable c'est elle qui s'occupe des erreurs sérieuses mais qui ne sont pas intéressantes à traiter (Ex : erreur hardware)
- **UNCHECKED EXCEPTION** car elle ne fait pas l'objet d'un contrôle de la part du compilateur. - La classe Exception regroupe tout ce qui est intéressant à traiter (Ex : erreur fichier), une exception créée par l'utilisateur doit toujours dériver de cette classe
- **CHECKED EXCEPTION** car elle fait l'objet d'un contrôle de la part du compilateur. - 3 Stratégies sont possibles quand une méthode fait face à une exception :
 - o Elle la traite en fournissant un gestionnaire d'erreur (catch)
 - o Elle la capte mais dans le but de la relancer (throw)
 - o Elle ne la traite pas, mais implicitement elle la relance
 - *Premier exemple* : Créer une classe qui extends la classe exception, créer une classe qui utilise une méthode qui utilise cette exception(throw), créer du code qui utilise cette méthode (try-catch)

5. Quels sont les éléments de portabilité apportés par la librairie AWT ? Il y en a au moins trois ...

- AWT (Abstract Window Toolkit) a le rôle de fournir les classes nécessaires à l'écriture d'une application comportant une interface graphique. Ces classes font partie d'une hiérarchie dont la classe de base est Component, elle hérite elle bien évidemment de la classe Object.

La classe Component est abstraite et représente un objet quelconque qui a une représentation graphique qui peut être affichée à l'écran et interagir avec l'utilisateur.

Tous ces objets d'interface sont indépendants de la plateforme hôte → portabilité. Pour se faire AWT utilise des objets qui sont les doubles (ou peers) des composants initiaux. Ces

doubles sont spécifiques à chaque environnement, ce sont ces objets qui déterminent l'aspect du contrôleur. On ne peut pas utiliser les coordonnées graphiques, Java a donc un gestionnaire de mise en pages.

6. En utilisant l'exemple de la réaction à l'appui sur un (J)Button, expliquer le mécanisme de traitement d'un événement graphique selon le principe "event-listeners".

Les événements sont représentés comme des instances d'une classe appartenant à une hiérarchie de classe d'événement, dont `EventObject`. L'objet instanciant l'une de ces classes contiendra toutes les informations concernant l'événement détecté ainsi que l'identification de sa source. Il existe différents listeners, ce sont des interfaces, il faut les implémenter à bon escient.

Exemple : Appuyer sur un bouton, ce bouton sera traité par un objet réalisant un `actionlistener`, la méthode de cette interface est `actionperformed`.

Une application désirant traiter l'événement « Appuyer bouton » devra :

- Instancier un objet implémentant un `actionlistener` - Enregistrer cet objet comme un listener de l'action (`addActionListener`)

Quand on appuie sur le bouton

- Un objet `ActionEvent` est instancié - La réaction de l'objet réalisant l'`actionlistener` sera d'invoquer sa méthode `actionperformed`

Remarque : Le GUI reste bloqué et on peut enregistrer plusieurs listeners sur le même événement.

Partie 4

1(A compléter). Expliquer le mécanisme général de traitement d'un événement graphique selon le principe "event-listeners". Décrire en toute généralité les hiérarchies de classes (en particulier: pourquoi EventObject n'a qu'une seule classe fille AWTEvent ?) et d'interfaces fournies par le JDK. Comparer les événements "Appui sur bouton", "Action sur une Fenêtre" et "Usage de la souris" (les points communs et les différences). Qu'est-ce qu'un adapter et comment l'utiliser ?

Les événements sont représentés comme des instances d'une classe appartenant à une hiérarchie de classe d'événement, dont EventObject. L'objet instanciant l'une de ces classes contiendra toutes les informations concernant l'événement détecté ainsi que l'identification de sa source. Il existe différents listeners, ce sont des interfaces, il faut les implémenter à bon escient.

Exemple : Appuyer sur un bouton, ce bouton sera traité par un objet réalisant un ActionListener, la méthode de cette interface est actionPerformed.

Une application désirant traiter l'événement « Appuyer bouton » devra :

- Instancier un objet implémentant un ActionListener
- Enregistrer cet objet comme un listener de l'action (addActionListener)

Quand on appuie sur le bouton

- Un objet ActionEvent est instancié
- La réaction de l'objet réalisant l'ActionListener sera d'invoquer sa méthode actionPerformed

Remarque : Le GUI reste bloqué et on peut enregistrer plusieurs listeners sur le même événement.

2. Expliquer les spécificités de la librairie Swing, notamment par rapport à la librairie AWT. Quels sont cependant les points communs ?

AWT - Composants Java pour GUIs avec une partie native ! Car avec les classes peer, ses composants utilisent les éléments d'interfaces natifs de la plateforme hôte. Ces composants sont qualifiés de « lourds »

Swing - Composants Java pour GUIs posant ses bases sur AWT mais qui est complètement écrit en Java et qui reposent simplement sur les primitives graphiques de la plateforme hôte. Ces composants sont généralement plus « complets » que les composants AWT. Mais la vraie différence c'est qu'ici on peut gérer le « look and feel » autrement dit l'apparence visuelle des composants. Elle peut varier en fonction des plateformes hôtes ou, au contraire, rester la même d'une plate-forme à une autre. Ces composants sont qualifiés de « légers »

AWT et Swing font tout les deux parties des JFC

Tous les composants Swing héritent de la classe JComponent. En fait, celle-ci est elle-même une descendante de la classe java.awt.Container, si bien que tous les composants Swing sont des containers (au sens GUI du terme) et sont donc susceptibles de se voir ajouter des composants dans un ordre fixé par un gestionnaire de mise en page. Par rapport à AWT, il y a un complément quant à l'ajout d'un gestionnaire de mise en page et de composant sur une tel fenêtre. Swing utilise les gestionnaires de mise en page de l'AWT.

3. Expliquer, avec le code explicite correspondant, les deux techniques qui permettent d'utiliser dans une fenêtre d'application un résultat obtenu au moyen d'une boîte de dialogue que cette fenêtre a fait apparaître.

4. Expliquer la notion de modèle dans les composants Swing. En particulier, comment les utiliser pour une JComboBox, une JList et une JTable ? [à compléter avec le cours suivant]

Un modèle est une espèce de template paramétrables pour les composants graphiques, c'est le langage Smalltalk qui introduisit le modèle appelé MVC, pour Model-View-Controller. Comme son nom l'indique, le paradigme de ce modèle est de séparer une application GUI en trois groupes d'objets : Les objets de types modèles, vues et contrôleur.

Swing propose en fait une variante simplifiée du modèle MVC. En effet, tout composant Swing contient en fait Un modèle [model], qui gère les données internes concernant l'état du composant et un responsable UI.

L'objectif de ce regroupement est de privilégier un concept Document-View cher à la P.O.O, un même ensemble de données peut se voir associer plusieurs vues.

Concrètement, tous les composants Swing héritent de la classe JComponent. Celle-ci ne comporte pas de modèle, ce sera donc aux classes dérivées d'en fournir un, spécifique au composant considéré et possède néanmoins une variable membre. Un modèle contient les informations qui déterminent l'aspect et le comportement du composant graphique.

```
//Pour une jlist
DefaultListModel model_list = new DefaultListModel();
model_list.addElement("Bou");
model_list.addElement("Margaux");
model_list.addElement("Lenny");
this.jList1.setModel(model_list);

//Pour une JComboBox
DefaultComboBoxModel model_combobox = new DefaultComboBoxModel();
model_combobox.addElement("Bou");
model_combobox.addElement("Margaux");
model_combobox.addElement("Lenny");
this.jComboBox1.setModel(model_combobox);

//Pour une Jtable
Vector vector = new Vector();
vector.add("Lenny");
vector.add("Tom");
vector.add("Anthony");
DefaultTableModel model_table = (DefaultTableModel) this.jTable1.getModel();
model_table.addRow(vector);
```

Partie 5

1. Expliquer ce qu'est un objet Enumeration et comment on l'utilise pour un Vector et pour un StringTokenizer.

La plupart des collections créent une ou plusieurs variables privées qui implémentent un interface nommé Enumeration. Comme son nom l'indique, celui-ci définit un protocole de parcours séquentiel; il déclare en effet les méthodes :

```
Public abstract boolean hasMoreElements()
```

Cette méthode teste s'il existe encore des éléments à itérer.

```
Public abstract Object nextElement()
```

Retourne l'élément suivant de l'énumération - lance l'exception NoSuchElementException s'il n'y a plus d'éléments.

On peut implémenter soi-même une telle énumération. Mais très souvent, on se contente d'utiliser celles qui sont toutes faites pour agir sur les classes collections.

```
StringTokenizer st = new StringTokenizer(textbox_mot_cle.getText(),"- /");  
while (st.hasMoreTokens())comboBox_mot_cle.addItem(st.nextToken());
```

```
Vector<Object> vecteur = new Vector<>();  
  
vecteur.add("Lenny");  
vecteur.add("Tom");  
vecteur.add("Anthony");  
  
Enumeration e = vecteur.elements();  
while(e.hasMoreElements())System.out.println("Prénom : " + e.nextElement());
```

2. Expliquer ce qu'est la classe Hashtable et donner des exemples d'usage classique de cette classe. En particulier, décrire ce qu'est la classe Properties (Hashtable ? mais avec quelles restrictions ?). [à compléter avec le chapitre consacré aux flux]

- Hashtable est une classe qui propose une table appliquant les principes du hashcoding et implémentant l'interface Map (Interface de collection qui gère les données avec un système de clé - valeur).
- Les properties contiennent les mêmes ensemble de couple clé-valeur. Quand on crée un tel objet, on peut lui fournir un fichier de properties ou on demande les caractéristiques du système par la méthode `System Public static Properties getProperties ()`.
- Attention ! Comme properties hérite de hashtable, les méthodes `put` et `putAll` peuvent être utilisées mais déconseillé ! Elles permettent à l'appelant d'insérer des entrées dont les clés ou les valeurs ne sont pas des chaînes. Les objets properties sont alors compromis. Les propriétés ne stockent que les valeurs sous forme de chaînes.

3. Expliquer comment on peut utiliser les Properties fournies par la JVM pour augmenter la portabilité des applications Java [à compléter avec le chapitre consacré aux flux]

On trouve dans le package `java.util` la classe `Properties` qui contient le même ensemble de couples clé/valeur. Il s'agit donc d'une structure de données de type dictionnaire et, de fait, la classe en question dérive de `java.util.Dictionary` par l'intermédiaire de `java.util.Hashtable` et implémente l'interface `Map`.

Lorsque l'on crée un tel objet, on peut lui fournir un fichier de propriétés ou demander les caractéristiques du système par la méthode `Public static Properties getProperties()` de `System`. La méthode `list()` permet d'afficher le contenu obtenu sur le flux de sortie précisé tandis que la méthode `Public String getProperty(String key)`.

Permet d'obtenir la valeur de la propriété dont on passe le nom comme paramètre (la méthode publique de `Properties` `propertyNames()` permet d'obtenir, sous forme d'`Enumeration`, le nom des propriétés disponibles). Ce qui peut donner :

TestSystem.java (2)

```
import java.util.*;

public class TestSystem
{
    public static void main(String args[])
    {
        Properties propCur = new Properties(System.getProperties());
        System.out.println("--- Liste des propriétés du système (list) ---");
        propCur.list(System.out);

        System.out.println("--- Liste des propriétés du système (enumeration) ---");
        for (Enumeration e = propCur.propertyNames(); e.hasMoreElements() ;)
        {
            String prop = (String)e.nextElement();
            System.out.println(prop + " --> " + propCur.getProperty(prop));
        }
    }
}
```

4. Expliquer comment on peut créer et afficher une date correctement et en accord avec les habitudes de l'utilisateur. En particulier, décrire les différentes classes, classes abstraites et interfaces qui interviennent dans ces manipulations (diagramme UML). En particulier, quelles sont les éléments de portabilité qui y interviennent ?

- La classe `Date` définie dans le package `java.util` et héritée d'`Object`, à l'occasion de la construction d'une applet affichant l'heure. En fait, celle-ci suffit largement pour de simples références aux dates et heures GMT. Le constructeur sans argument public `Date()` prend en fait la date système.

Il existe d'autre constructeur d'initialisation et des méthodes permettant extraire les éléments d'une date.

- La classe abstraite `Calendar` permet de déterminer la manière d'enregistrer et de manipuler les données temporelles, Cette classe possède bien entendu des constructeurs mais on peut plutôt utiliser :
 - La classe `GregorianCalendar`, qui implémente le calendrier grégorien occidental courant
 - La méthode de classe public static synchronized `Calendar getInstance()` Cette méthode fournit un objet calendrier la mieux adapté selon la plateforme, une telle méthode, qui fabrique l'objet le mieux adapté au contexte d'exécution s'appelle une méthode factory.
- Exemple :

```
public final Date getTime()
```

```
Calendar c = Calendar.getInstance();  
System.out.println("Date calendrier = " + c.getTime());
```

donne :

```
| Date calendrier = Tue Jan 04 18:35:33 CET 2005
```

La classe `Locale` permet d'encapsuler tout ce qui caractérise une région géographique en termes de pays, de langue, d'habitudes d'indication d'heure.

Elle possède un constructeur assez prévisible : `Public Locale(String langue, String pays)`. Elle contient des constantes de classes qui sont des instances de locale qui permettent de spécifier le contexte régionale.

La classe `DateFormat` du package `java.text` et héritée indirectement d'`Object` en passant par une classe intermédiaire `Format`. Son rôle est essentiellement de gérer le format d'affichage d'une date en tenant compte du contexte régional. Une méthode particulièrement

intéressante est la méthode de classe : `Public static final DateFormat
getDateTimeInstance(int dateStyle, int timeStyle, Locale aLocale)`

```
Date maintenant = new Date();  
String maDate = DateFormat.  
    getDateTimeInstance(DateFormat.FULL,DateFormat.FULL,Locale.US).  
    format(maintenant);  
System.out.println("Date et heure = " + maDate);  
  
maDate = DateFormat.  
    getDateTimeInstance(DateFormat.FULL,DateFormat.FULL,Locale.FRANCE).  
    format(maintenant);  
System.out.println("Date et heure = " + maDate);
```

ce qui nous donne :

```
Date et heure = Tuesday, January 4, 2005 6:44:09 PM CET  
Date et heure = mardi 4 janvier 2005 18 h 44 CET
```

On peut remarquer que, cette fois, l'heure est correcte pour Seraing ...

5. Expliquer ce qu'est une méthode factory. En quoi son utilisation contribue-t-elle à la portabilité ? Expliquer un exemple de factory sans paramètre et de factory avec paramètre(s).

Une **méthode factory** est une méthode :

- Statique capable d'instancier un objet d'une certaine classe ou, surtout, implémentant un certain interface, en fonction des paramètres qu'on lui passe.
- Ces paramètres ne sont pas du tout ceux que l'on passerait a un constructeur
- C'est le constructeur par défaut qui sera utilisé et la valeur des variables membres sera fixée au moyen des méthodes set

Une **méthode factory** permet d'instancier une classe :

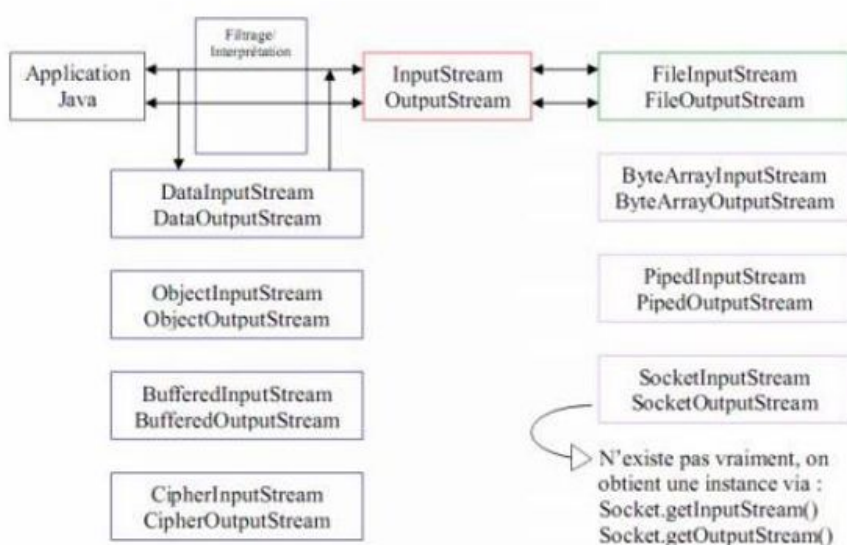
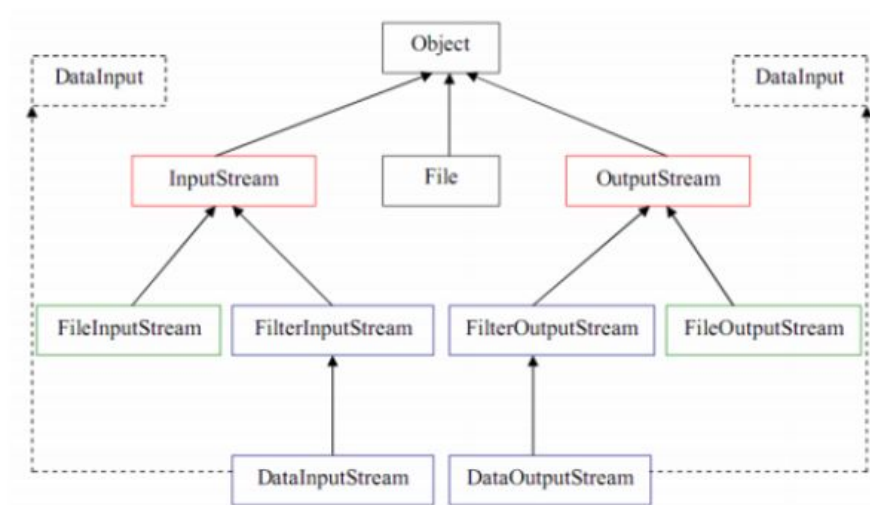
- Au moyen du chargeur de classe spécifié par le premier paramètres (null si c'est celui par défaut)
- De récupérer l'objet sérialisé dans la ressource spécifiée en second paramètres
- En cas d'échec seulement, elle recherche une classe ayant ce nom (précédé du nom du package) et crée une instance avec les valeurs par défaut
- De manière dynamique : on peut choisir, au moment de l'exécution, la classe à instancier, selon les critères qui ne peuvent être évalués qu'à ce moment. Ce qui ne serait pas possible avec un constructeur, dont le nom doit forcément être connu au moment de la compilation.

6. Expliquer comment on peut gérer un double-clic sur une JList en utilisant le paradigme Event-Listener-Adapter de AWT/Swing. Ne pas oublier de compléter la question 4 du questionnaire précédent (4):

Partie 6

1. Expliquer le modèle de flux à n couches de Java permettant à une application d'écrire ou de lire des données typées; en particulier, combien y a-t-il de couches au minimum et au maximum? En quoi ce modèle est-il générique par rapport aux diverses sources de données ?

Les flux sont des abstractions qui représentent la gestion d'un flot de bytes en entrée ou en sortie. Autrement dit, il s'agit de classes qui encapsulent les mécanismes, parfois compliqués, de ces entrées/sorties. Du point de vue de la programmation, un flux peut être vu comme une suite de données dont la longueur est indéterminée.



- Les classes de bases : InputStream et OutputStream sont 2 classes abstraites qui considèrent les entrées et sorties comme des suites de bytes.
 - o InputStream comporte une interface minimale et une implémentation minimale des flux d'entrée
- OutputStream comporte une interface minimale et une implémentation minimale des flux de sortie - Les flux bas niveau :
- File est une classe qui permet de créer un objet instanciant l'identification du fichier indépendamment de l'implémentation hôte. Ses méthodes encapsulent tout ce qui dépend du système de fichiers utilisés. Cette classe réalise donc l'interface entre le traitement classiques de fichiers peu importe l'environnement qui les héberge et les caractéristiques particulières de cet environnement. File n'est pas un flux.
 - o FileInputStream est une classe dérivée d'InputStream et chargée de la gestion des fichiers d'entrée. Son constructeur réclame l'objet File sur lequel le flux s'appuiera. Cette classe permet donc le traitement en entrée au niveau des bytes. La méthode read lit des bytes sans la moindre interprétation.
 - o FileOutputStream est une classe dérivée d'OutputStream et chargée de la gestion des fichiers de sortie. Son constructeur permet d'ouvrir un fichier en création ou en ajout (grâce au boolean donné en paramètre)
- Les flux d'interprétation : On trouve une classe dérivée de InputStream nommée FilterInputStream. Cette classe met en place un mécanisme additionnels qui permettront à ses sous-classes de réaliser l'interprétation.
 - o DataInputStream qui permet de lire un fichier texte considéré ligne par ligne
 - o DataOutputStream fournit les méthodes plus ou moins équivalentes à celles de DataInputStream.
- Les deux classes DataInputStream et DataOutputStream sont parmi les plus importantes de tout l'ensemble des flux Java
- Les flux de traitement complémentaires : Afin de sauvegarder des objets sur un support permanent, on parle encore de sérialisation ou d'objet persistant, on utilise les classes ObjectOutputStream et ObjectInputStream, dérivées respectivement de OutputStream et d'InputStream. Elle permet de sauvegarder et de charger n'importe quel objet.

2. Expliquer comment Java gère la portabilité par rapport au système de fichiers de la machine hôte en utilisant la classe File et les Properties du système.

Portabilité :

- Le code généré par le compilateur Java est exécutable sur tout environnement disposant de l'interpréteur Java approprié - Le code généré par un compilateur Java est encore appelé du bytecode, ses instructions sont exprimées au niveau le plus bas possible sans devenir dépendant de la machine d'exécution. - C'est le rôle de la machine virtuelle de transformer le bytecode en du code compréhensible par la machine hôte. - L'intérêt est de développer des applications indépendamment de la plateforme sur laquelle elles fonctionnent effectivement
- La classe System est une classe dite «final». Une classe «final» ne peut être dérivée. Des raisons de désigne peuvent justifier ce choix, mais également de sécurité. Cette

classe fournit le mécanisme rendant l'indépendance entre les applications et la plateforme sur laquelle elles fonctionnent. Elle fournit :

- Les entrées/sorties standards - Les caractéristiques du système et diverses autres informations sous forme encapsulée.

Toutes les variables membres et les méthodes de cette classe sont statiques. Les trois flux standards in, out et error sont des variables de classe. Il est aussi possible d'obtenir un certain nombre d'informations utiles sur le système au moyen de la méthode public static String getProperty(String key). On repousse donc au moment de l'exécution le choix de la classe ; c'est la factory

File est une classe qui permet de créer un objet instanciant l'identification du fichier indépendamment de l'implémentation hôte. Ses méthodes encapsulent tout ce qui dépend du système de fichier utilisés. Cette classe réalise donc l'interface entre le traitement classiques de fichiers peu importe l'environnement qui les héberge et les caractéristiques particulières de cet environnement. Ses méthodes fournissent tout naturellement les renseignements système sur le fichier considéré : Public String getName() Public String getPath() Etc...

3. Expliquer le mécanisme de sérialisation/désérialisation de Java assurant la persistance des objets. En particulier, que trouve-t-on dans un fichier ou dans une capture de sniffer après plusieurs écritures d'objets instance d'une même classe ?

- La **sérialisation** est un processus de mémorisation de l'état d'un objet (ou d'un groupe d'objets) dans le but d'une récupération ultérieure. On parvient à un tel objectif en utilisant les classes ObjectOutputStream et ObjectInputStream, dérivées respectivement d'OutputStream et d'InputStream.
- La **sérialisation d'objet** implique d'utiliser une classe ObjectOutputStream, qui implémente l'interface ObjectOutput. Celui-ci, hérité de DataOutput, étend ce dernier des types primitifs aux objets en tous genres et aux tableaux. Méthodes importantes : Public final void writeObject (Object obj) throws IOException. Celle-ci mentionne dans le fichier la classe de l'objet qu'elle enregistre, sa signature et les valeurs des différentes variables membres d'instance. Il suffit donc d'enregistrer des objets de même type pour obtenir l'équivalent (mais seulement l'équivalent) d'un fichier séquentiel à enregistrements. Concrètement, si l'on dispose d'un tableau d'objets signalétiques
- La **désérialisation** d'objet implique d'utiliser la classe ObjectInputStream qui implémente l'interface ObjectInput (hérité de DataInput et qui en est une extension aux objets et aux tableaux). Méthodes importantes : Public final void ReadObject (Object obj) throws ... IOException. C'est cette dernière méthode qui nous intéresse, puisqu'elle permettra de relire le fichier en tenant compte des informations sur le type des données à lire. Inutile de dire qu'il est vital que les classes des objets utilisés pour l'écriture et la lecture soient rigoureusement les mêmes - L'interface Serializable

ne contient aucune méthode à implémenter ! En fait, l'intérêt pour une classe de déclarer qu'elle "implémente" cet interface est de signaler qu'elle désire activer le processus de sérialisation

4. Comment utiliser dans une application un fichier Properties comme fichier de configuration ?

L'usage de cette classe peut voir son utilité dans la sauvegarde de certains éléments d'une application comme la façon d'afficher des dates, etc. En et il ne serait pas judicieux de sauvegarder cela dans une base de données. Leur rôle est donc bien sûr de servir de fichier de configuration élémentaire. Un autre avantage de ce type de fichier est qu'il peut être lu par n'importe quel éditeur de texte, car sa composition est élémentaire : < nom propriété > <= ou :> < valeur de la propriété > l'extension . Lors de l'application des gestion de news, le fichier de configuration a servi a stocké les différent chemin d'accès des différents fichiers comme le fichiers des news, des logins, des ports etc.

5. Décrire le fonctionnement de l'utilitaire jar. En particulier, expliquer le rôle des clauses classiques courantes du fichier manifeste qui permettent d'exécuter une application délivrée sous forme de jar.

- L'utilitaire jar.exe (appartenant au JDK et se trouvant dans le répertoire bin du jdk) permet de rassembler dans un même fichier d'extensions jar un certain nombre d'autres fichiers qui sont typiquement (mais sans aucune limitation) :
 - Des fichiers bytescodes .class ;
 - Des fichiers .xml ou .html ;
 - Des fichiers images .gif ou .jpg - L'intérêt d'utiliser jar est que l'on peut ainsi produire un fichier :
 - Comprimé : il prend donc moins de place sur le disque ou sur le réseau ; le format de compression est le format ZIP ;
 - Sécurisé : il est possible d'y intégrer une signature électronique.
- Le rôle du fichier manifeste est de contenir des informations sur les fichiers encapsulés dans le jar. La nature de ces informations relève essentiellement Du déploiement dans le cas d'une application : il peut contenir des indications sur d'autres jars nécessaires à l'exécution de l'application (ces jars devront donc faire partie du jar principal) ;
 - De l'exécution, toujours dans le cas d'une application : il contient ainsi le nom de la classe qui contient la méthode main() qui fait démarrer le programme ;
 - De l'exécution, toujours dans le cas d'une application : il contient ainsi le nom de la classe qui contient la méthode main() qui fait démarrer le programme ;

Le fichier manifeste par défaut se contente de contenir la version de la spécification utilisée.

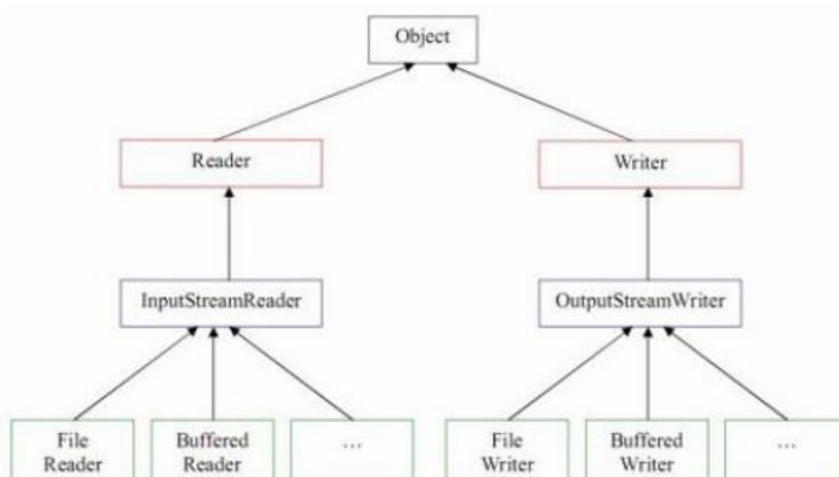
Partie 7

1. Expliquer le rôle des principales classes de flux qui permettent de gérer correctement les données de type caractères. Illustrer par le code qui lit un nom de fichier au clavier, qui écrit quelques phrases dans ce fichier, qui ferme le fichier puis qui le relit intégralement. Quelles sont les classes cachées qui expliquent une prise en charge correcte de ces caractères ?

On ne dirait pas à première vue mais il existe des nuances entre les flux orientés texte et les flux orientés bytes. Ces différences, qui sont basées sur la différence même entre des données "binaires" et "textes", justifient l'existence en Java d'une seconde hiérarchie orientée caractères : il s'agit de la famille des classes abstraites `Reader` et `Writer` du package `java.io`. Ces flux proposent des méthodes qui sont capables de lire ou d'écrire non plus de simples bytes, mais des caractères, avec le travail de décodage que cela implique. En ce sens, ce sont des flux plus évolués, puisqu'ils interprètent selon le contexte machine dans lequel ils travaillent.

Méthode pour l'écriture : `Public void write(char cbuf[]) throws IOException` (Ressemble fort à la méthode de la classe `OutputStream`)

Le seul complément est que l'on trouve en plus dans la classe `Writer` des méthodes utilisant des objets `String` : `Public void write(String str) throws IOException`. Ce qui est normal puisque les classes de type `OutputStream` ne connaissent que les bytes



ATTENTION SCHEMA FAUX : Buffered Read et writer hérite directement de reader et writer

Code qui lit un nom de fichier au clavier, qui écrit quelques phrases dans ce fichier, qui ferme le fichier puis qui le relit intégralement :

```

public static void main(String[] args) throws IOException
{
    String nomFichier;
    String texte;
    BufferedReader in;

    //Saisir pour savoir où se trouve le fichier
    System.out.print("Nom du fichier à afficher : ");
    in = new BufferedReader(new InputStreamReader(System.in));
    nomFichier = in.readLine();

    //Saisir
    System.out.print("Saisir du texte: ");
    in = new BufferedReader(new InputStreamReader(System.in));
    texte = in.readLine();

    //Ecriture
    System.out.print("Ecriture dans le fichier...");
    FileWriter f = new FileWriter(nomFichier, true);
    BufferedWriter bf = new BufferedWriter(f);

    bf.write(texte);
    bf.newLine();
    bf.close();

    //Lecture
    System.out.print("Lecture dans le fichier...");
    FileReader fr = new FileReader (nomFichier);
    BufferedReader br = new BufferedReader(fr);
    String s;
    while ((s=br.readLine()) != null)
        System.out.println(s);
}

```

2. Expliquer ce qu'est un Java Bean et décrire avec un exemple l'architecture de programmation basée sur un événement et qui comporte deux beans et leur container.

Les Java Beans sont des composants logiciels réutilisables, en fait des classes Java, qui possèdent des propriétés (en rapport avec les variables membres) et des fonctionnalités (matérialisées par ses méthodes). En assemblant (notamment avec un outil graphique) de tels composants, il est possible de construire une application Java gardant ses qualités habituelles : les beans, s'ils sont des unités de code fonctionnel et autonomes, peuvent en fait communiquer et se connecter entre eux pour former des applications complètes. Pour être un Java Bean, Un Bean doit proposer un certain nombre de services de base :

- L'indication de son état
- La possibilité pour un développeur, de prendre connaissance des propriétés et des méthodes du composant et de les configurer.
- La gestion des événements qu'il est capable de gérer et de répercuter au niveau des autres composants - Il doit pouvoir assurer la sérialisation (Persistance)
- L'existence d'un mécanisme d'appel dynamique
- L'existence d'un mécanisme de réception des notifications d'événements par d'autres composants

De manière plus pragmatique, on pourrait dire qu'un JavaBean est un morceau de code compilé :

- Dont la source a été écrite exclusivement en Java et est donc une classe

- Qui peut être configuré par la définition de ses propriétés
- Qui possède un constructeur par défaut
- Qui est susceptible de générer des événements qui permettront de faire savoir aux autres acteurs de l'environnement que «quelque chose» s'est passée. Donner un exemple de Bean.

3. Expliquer le fonctionnement et l'intérêt de la factory instantiate() des Java Beans (résultat, paramètres, exceptions susceptible d'être lancées, ...).

L'instanciation d'un bean ne se fait jamais par appel d'un constructeur. La classe Bean propose la méthode public static Object instantiate (ClassLoader cls, String beanName) throws IOException, ClassNotFoundException

Partie 8

1. Dans le contexte des Java Beans, qu'entend-on par "propriété liée" et "propriété contrainte" ? Expliquer le fonctionnement des outils de gestion de ces propriétés particulières.

- Une "propriété liée" est une propriété qui donne lieu à l'émission d'un événement lorsque sa valeur change.
- La classe PropertyChangeSupport permet de gérer les événements associés aux changement de propriété d'un bean.
 - Le bean considéré possède un tel objet comme variable membre (appelons-la GestProp) et lui délègue cette partie du travail. Son constructeur est Public PropertyChangeSupport(Object sourceBean).
 - La méthode public void firePropertyChange (String propertyName, Object oldValue, Object newValue) permet de prévenir tous les objets intéressés
 - Les objets intéressés par le changement de propriété devront se faire enregistrer auprès de l'objet visé en utilisant la méthode public synchronized void addPropertyChangeListener(PropertyChangeListener listener)

- Une propriété contrainte est une propriété qui ne peut être modifiée que si la modification envisagée est validée par tous les beans cibles.
- La classe `VetoableChangeSupport` permet de gérer les événements associés aux changements validés de propriétés d'un bean.
 - Le bean considéré possède un tel objet comme variable membre (appelons-la `VetoProp`) et lui délègue cette partie du travail. Son constructeur est public `VetoableChangeSupport(Object sourceBean)`
 - La méthode public void `fireVetoableChange (String propertyName, Object oldValue, Object newValue)` throws `PropertyVetoException` permet de prévenir tous les objets intéressés
 - La différence par rapport à `PropertyChangeSupport` est qu'elle est susceptible de lancer une exception `PropertyVetoException` : lorsque c'est effectivement le cas, cela signifie évidemment que l'un des beans cibles a refusé la modification – c'est donc ce bean cible qui a lancé l'exception.
 - Les objets intéressés par le changement de propriété devront se faire enregistrer auprès de l'objet visé en utilisant la méthode public synchronized void `addVetoableChangeListener(VetoableChangeListener listener)`

2. Expliquer ce qu'est en toute généralité ce qu'est une annotation en Java. Illustrer en expliquant l'usage des annotations `@Override` et `@SuppressWarnings`. Pourquoi parle-t-on aussi de "méta-annotations" ?

- Une annotation est un symbole constituée du caractère «@» et d'un identifiant destiné à marquer un élément du code (une classe, une variable membre, etc.) en le précédant.

En toute généralité, elle permet d'ajouter à cet élément une propriété particulière. Plus précisément, elle va, selon sa nature, influencer la compilation ou l'exécution ou encore tout processus qui, directement ou indirectement, va traiter le code Java considéré. Les annotations standard sont (`Override`, `Deprecated` et `SuppressWarnings`) - Le développeur peut définir ses propres annotations : une telle définition est en fait celle d'une Interface, mise à part la présence du symbole «@» devant le mot réservé `interface`, indiquant que l'usage de l'interface sera un peu particulier, puisque qualifiant un autre élément du code source.

- **@Override** Cette annotation ne peut marquer, au sein d'une classe dérivée, qu'une méthode supposée redéfinir la méthode du même nom de la classe mère. Si il s'avère que ce n'est pas le cas (à cause par exemple d'une simple faute de frappe dans le nom de la méthode dérivée), le compilateur diagnostiquera une erreur
- L'annotation **@SuppressWarnings** signale au contraire au compilateur qu'il ne doit pas afficher certains warnings : la nature de ces warnings constitue un attribut de l'annotation; sa valeur est construite avec des symboles comme "deprecation" ou "unchecked".